

[← Go Back](#)

Hardware

Nicla Voice



Tutorials

Door Intruder Detector
Using ML with the Nicla
Voice

Audio Analysis with
Machine Learning and the
Nicla Voice

Glass-Breaking Detection
Using Edge Impulse & Nicla
Voice

**Motion Detection with
Nicla Voice and Machine
Learning Tools**

Nicla Voice User Manual

[Home](#) / [Hardware](#) / [Nicla Voice](#) /

Motion Detection with Nicla Voice and Machine Learning Tools

Motion Detection with Nicla Voice and Machine Learning Tools

Learn how to create a simple motion detection application with the Arduino® Nicla Voice and the Arduino Cloud Machine Learning Tools powered by Edge Impulse®.

Author José Bagur,
Christopher Mendez,
and Benjamin Dannegård
Last revision · 07/17/2024

ON THIS PAGE

Overview

Goals

Hardware and Software Requirements +

Motion Detection and TinyML

Instructions +

Conclusion +

Overview

The Nicla Voice is a versatile board that can be used for various projects and applications. With its onboard Inertial Measurement Unit (IMU) and Tiny Machine Learning (TinyML), the Nicla Voice board provides an ideal device for the application described in this tutorial: **a simple motion detection application.**



The Nicla Voice board

The simple motion detection application will be based on a TinyML model developed with the [Arduino](#)

[Help](#)

platform in developing intelligent devices and a powerful facilitator for TinyML applications. With the Machine Learning Tools, we will capture data from your Nicla's Voice onboard IMU, create and train a TinyML model specifically designed for motion detection, and deploy it into the board for testing and validation. By the end of this tutorial, you will have a fully functional motion detection application running on your Nicla Voice board and a solid foundation in applying TinyML to real-world applications.

Goals

Understand the key features and capabilities of the Nicla Voice board.

Set up a Nicla Voice board and prepare it for developing of a motion detection application.

Become familiar with the Machine Learning Tools integration of the Arduino Cloud, its interface, and its key features.

Collect motion data from the Nicla Voice board's onboard IMU using the Arduino Cloud Machine Learning Tools.

Develop, train, and validate a TinyML model designed for motion detection using the Machine Learning Tools.

Deploy the trained TinyML model onto the Nicla Voice board and test its real-time motion detection capabilities.

Gain practical experience and foundational knowledge in developing TinyML applications for real-world scenarios.

Hardware and Software Requirements

Hardware Requirements

[Nicla Voice](#) (x1)

[Micro-USB](#) cable (x1)

Software Requirements

[Arduino IDE 2.0+](#), or [Arduino Cloud Editor](#)

To create custom Machine Learning models, the integrated [Machine Learning Tools](#) of the [Arduino Cloud](#) are needed. In case you do not have an Arduino Cloud account, you will need to create one first.

Motion Detection and TinyML

Motion detection technology, which identifies and measures physical movement, is vital in many fields. The precision and scope of motion detection can be significantly improved by using an IMU, an advanced device designed to capture the force and angular velocity. This enhanced capability is important in real-world applications such as wearable fitness trackers, vehicle safety and navigation systems, and robotic movement controls.

Within this context, TinyML emerges as a transformative new force. As a specialized branch of Machine

developed for devices with limited computational power, such as microcontrollers. Despite its compact nature, TinyML brings robust Machine Learning capabilities without imposing significant power or computational burdens. Its essence lies in converting raw data into actionable insights. A TinyML model can discern specific movements and irregularities by interpreting patterns from the IMU data, empowering a device to interact dynamically with its environment.

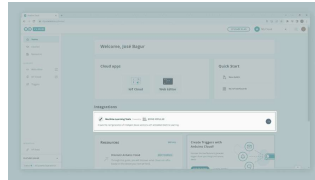
For the development process in this tutorial, we'll employ the Machine Learning Tools, an great platform for TinyML application development. This platform features an interesting suite of tools for data acquisition, model design, model training, and deploying our motion detection model on the Nicla Voice board. This approach facilitates the design and implementation of complex models directly on the board. By combining the capabilities of the Nicla Voice board, the advantages of an IMU-based motion detection model, and the versatility of a platform like the Arduino Cloud, this tutorial aims to provide an exhaustive perspective on constructing tangible real-world applications.

Instructions

Creating an Arduino Cloud Account

First, if you still need a Cloud account, please create one [here](#). You can then access the [Machine](#)

Cloud account, as shown in the image below.

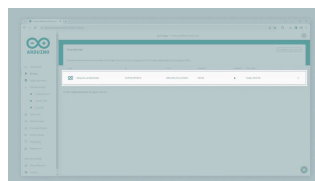


Cloud welcome page

With your account set up and running, create a new project. With your new project created, we are ready to capture data for your motion detection model.

IMU Data Capture with the Machine Learning Tools®

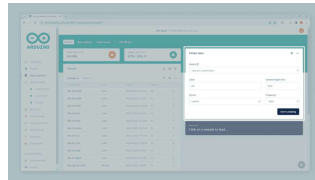
We can capture data from the onboard IMU of the Nicla Board using the Machine Learning Tools. To do this, check out the following [article](#) from Edge Impulse® documentation. Follow the steps described in the article to connect your Nicla Voice board to the Machine Learning Tools; when your board is connected, under the **Devices** tab on the left side of the window, you should see your it as shown in the image below (green dot):



Device connected to the Machine Learning Tools

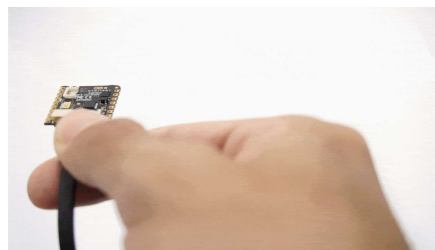
You can now start collecting IMU data with your Nicla Voice board connected to your Cloud account via its Machine Learning Tools

Tools, go to the **Data acquisition** tab on the left side of the window. **Data acquisition** is the tool from the Machine Learning Tools where IMU data can be sampled and stored. To sample new data, go to the **Collect data** window.

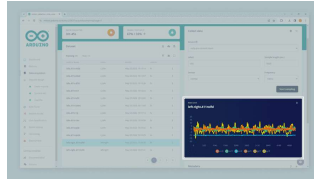


Data collection on the Machine Learning Tools

Select your Nicla Voice board, set the label to `left-right`, the sample length to `5000` ms, the sensor to `Inertial`, and the frequency to `100Hz`. This configuration means that data from the onboard IMU of your Nicla Voice board will be sampled for five seconds (5000 ms) and stored with the `left-right` label (labels can be edited later if needed). Click the **Start sampling** button and move your Nicla Voice board from left to right, as shown below.

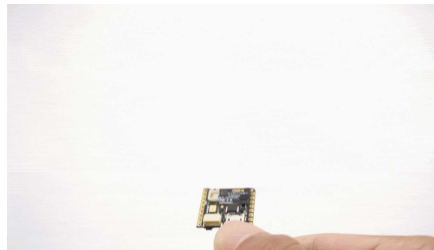


After the sampling and storing process is finished, you should see a new line under the **Collected data** window; when the data is selected, the raw sampled data is graphed as shown in the image below.



Data collection on the
Machine Learning Tools

Let's repeat the process but for a different type of movement. Set the label to `up-down`, the sample length to `5000` ms, the sensor to `Inertial`, and the frequency to `100Hz` (same as before). Click the **Start sampling** button and move your Niclea Voice board up and down, as shown below.



Now it's time to build the **dataset** for the motion detection application using the following **three classes**:

`left-right`: horizontal movements, from left to right.

`up-down`: vertical movements, starting up and going down.

`z_idle`: random movements that are not from left to right or starting up and going down.

The Syntiant NDP processors require a negative class on which no predictions will occur; **i** in our application, this is the `z_idle` class. Make sure the negative class name is last in alphabetical order.

Split your data into **training, test,** and **validation** sets: a common split

testing, and 15% for validation. For demonstration purposes, record around **four minutes** of data per class; in this case, you need to allocate approximately the following amounts of data for each set:

Training: 2.8 minutes (70% of 4 minutes).

Testing: 0.6 minutes (15% of 4 minutes).

Validation: 0.6 minutes (15% of 4 minutes).

The number of samples required for a motion detection model depends on several factors. There is no one-size-fits-all answer, but some general guidelines to determine the appropriate sample size are the following:

Data complexity: With complex patterns, larger sample size.

Type of motion detection technique: Supervised methods require more samples than unsupervised methods.

Model performance: More data is only sometimes better.

The rarity of movements: A larger sample size is needed if anomalies are rare events.

As a starting point, consider that you will need more samples, **at least several hundred to a few thousand**, for a more robust model.

Impulse Design

With the training dataset captured, you can design an **impulse**; an impulse takes raw data, uses signal processing to extract features, and then uses a learning block to classify new data. Signal processing blocks

data easier to process, while learning blocks learn from past experiences.

In this tutorial, we are going to use the following processing blocks:

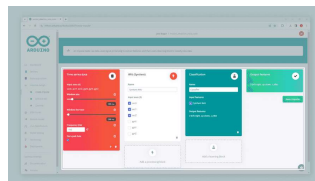
IMU Syntiant processing

block: This block prescales raw data to 8-bit values to match Nicla's Voice NDP processor input requirements.

Classification learning block:

This block takes the generated features and learns to distinguish between different classes (`left-right`, `up-down`, or `z-idle`).

To set both blocks, navigate to the **Create impulse** tab, select the window size to `600` ms, increase it to `200`, add the `IMU Syntiant` and `Classification` blocks, and click the **Save impulse** button. Your impulse should be as shown below:



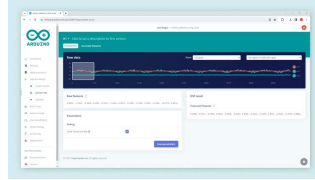
Designed Impulse with its learning and processing blocks

The Syntiant NDP101 processor requires the number of generated features to be divisible by four. In our example, we have three axes sampled at 100 Hz with a 600 ms window leading to 180 (60x3) features divisible by four.

IMU Syntiant Block Configuration

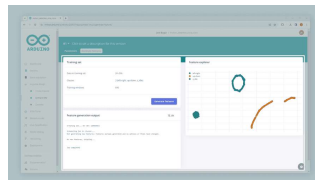
To configure the **IMU Syntiant**

raw data on top of the window (you can select other files via the drop-down menu) and the processed features on the right.



Syntiant IMU processing parameters configuration

Click the **Scale 16 bits to 8 bits** option; this converts your raw data to 8-bit and normalizes it to the $[-1, 1]$ range. Then, click the **Save parameters** button to send you to the **Feature generation** window. Click the **Generate features** button on that window to start the feature generation process.



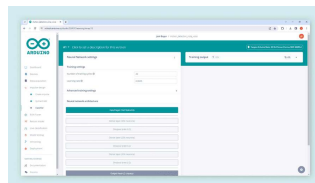
Syntiant IMU feature generation process

After a while, the **Feature explorer** window will load. This screen plots all the extracted features against all the generated windows, as shown in the image above; you can use this graph to compare your complete data set. A good rule of thumb is that **if you can visually separate the data on several axes, then the Machine Learning model will also be able to do so.**

Classification Block Configuration

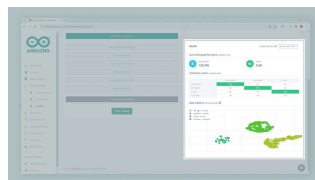
Now, it's time to start training a neural network. Neural networks are algorithms that can learn to recognize patterns in their training data, for example, certain types of movements. The neural network you will train will take the processing block features as input and try to map it to one of the three classes of our model (left-right , up-down , or z-idle).

Click on the **Classifier** tab, you'll see the following window:



Neural network settings window

Click the **Start training**. When it's complete, you'll see the **Model** performance window appear at the bottom-right of the window:



Model performance window

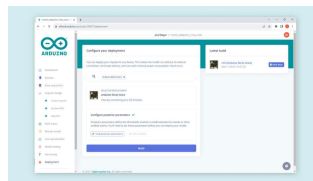
From the **Model** performance window, we can see the **model's accuracy**. Accuracy refers to the percentage of windows of movement that were correctly classified; the higher the number, the better. However, **an accuracy of 100% is unlikely and often a sign that your model has overfit the training data**. For most applications, an

confusion matrix shows the balance of correctly versus incorrectly classified.

Now that you have trained a neural network with the Machine Learning Tools, we are ready to deploy and test it on your Nicla Voice board!

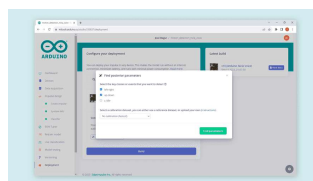
Deploying and Running an Impulse

It's time to test your model on your Nicla Voice board. Click on the **Deployment** tab, and then under the **Search deployment options** button, select the **Arduino Nicla Voice** option as shown below:



Configure your deployment window

Then, configure posterior parameters to define, for example, thresholds at which your model activates; posterior parameters are used to tune the precision of the neural network activations to minimize the False Rejection Rate and False Activation Rate. Click on the **Find posterior parameters** button and select the classes you want to detect; **make sure to uncheck the last class**, in this application, the `z-idle` class.

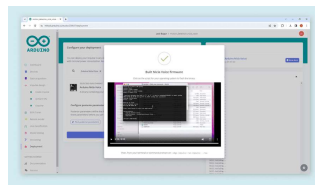


You can also deploy and run your impulse through a library generated by the Machine



Learning Tools for the Syntiant NDP101 processor of your Nicla Voice board. This library can also be built in the Deployment tab.

Click on the **Build** button. This will start a process on which a binary file containing your impulse is built; this file can then be run on your Nicla Voice board. After completing the building process, you'll be prompted to download a `.zip` file containing the binary file and save it on your computer.



Nicla Voice firmware generation

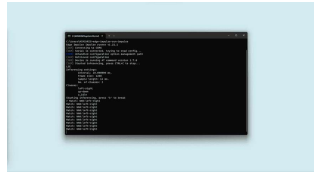
After unzipping the downloaded file, run the appropriate flashing script for your computer's operating system (Linux, Mac, or Windows) to flash your Nicla Voice board with the motion recognition model and associated firmware.

Running the Model

With your Nicla Voice board flashed, open a new terminal window and run the following command:

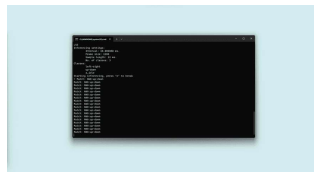
```
1 edge-impulse-run-impulse
```

This command will sample data from your Nicla's Voice onboard IMU, make inferences, and then classify the movement made on your board. You should see the following output in the terminal window when your Nicla Voice board is moved in horizontal movements, from left to right.



Impulse running in the
Nicla Voice board

When the board is moved in vertical movements, starting up and going down, you should see the following output in the terminal window:



Impulse running in the
Nicla Voice board

Conclusion

In this tutorial, you learned how to use your Nicla Voice board with the Machine Learning Tools integration of the Arduino Cloud to develop a simple motion detection application. Not only have you created a simple motion detection application, but you've also laid the groundwork for exploring more advanced and sophisticated applications in the future. You can access the

Interested in learning more about Machine Learning or learning about the Nicla Voice hardware and software capabilities? Check out the following tutorials and applications notes:

[Audio Analysis with Machine Learning and the Nicla Voice](#)

[Door Intruder Detector Using ML with the Nicla Voice](#)

Suggest changes	Need support?	License
The content on docs.arduino.cc is facilitated through a public GitHub repository. If you see anything wrong, you can edit this page here .	Help Center Ask the Arduino Forum Discover Arduino Discord	The Arduino documentation is licensed under the Creative Commons Attribution-Share Alike 4.0 license.

Was this article helpful?

